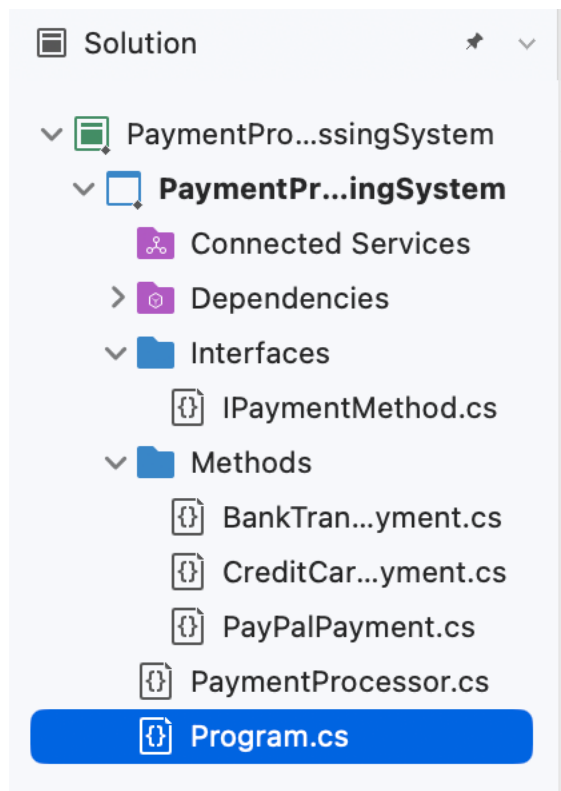




IPaymentMethod.cs

```
namespace PaymentProcessingSystem.Interfaces
{
    public interface IPaymentMethod
    {
        void ProcessPayment(decimal amount);
    }
}
```

BankTransferPayment.cs

```
using System;

using PaymentProcessingSystem.Interfaces;

namespace PaymentProcessingSystem.Methods
{
    public class BankTransferPayment : IPaymentMethod
    {
        public void ProcessPayment(decimal amount)
```

```

    {
        Console.WriteLine($"Processing bank transfer payment of {amount:C}");
    }
}
}

```

CreditCardPayment.cs

```

using System;

using PaymentProcessingSystem.Interfaces;

namespace PaymentProcessingSystem.Methods
{
    public class CreditCardPayment : IPaymentMethod
    {
        public void ProcessPayment(decimal amount)
        {
            Console.WriteLine($"Processing credit card payment of {amount:C}");
        }
    }
}

```

PayPalPayment.cs

```

using System;

using PaymentProcessingSystem.Interfaces;

namespace PaymentProcessingSystem.Methods
{
    public class PayPalPayment : IPaymentMethod
    {
        public void ProcessPayment(decimal amount)
        {
            Console.WriteLine($"Processing PayPal payment of {amount:C}");
        }
    }
}

```

PaymentProcessor.cs

```
using System;

using PaymentProcessingSystem.Interfaces;

namespace PaymentProcessingSystem
{
    public class PaymentProcessor
    {
        // Encapsulate the interface to achieve Loose Coupling (Decoupling)
        private readonly IPaymentMethod _paymentMethod;

        // DEPENDENCY INJECTION (DI):
        // Instead of creating a specific payment object inside this class,
        // I "inject" the dependency through the Constructor.
        public PaymentProcessor(IPaymentMethod paymentMethod)
        {
            // Validation: Ensure the injected object is not null
            _paymentMethod = paymentMethod ?? throw new ArgumentNullException(nameof(paymentMethod));
        }

        public void Process(decimal amount)
        {
            // Execute the payment logic without needing to know the specific implementation (OCP)
            _paymentMethod.ProcessPayment(amount);
        }
    }
}
```

Program.cs

```
using System;

using System.Collections.Generic;

using PaymentProcessingSystem.Interfaces;

using PaymentProcessingSystem.Methods;
```

```

namespace PaymentProcessingSystem
{
    class Program
    {
        static void Main(string[] args)
        {
            // Set encoding to UTF8 to support currency symbols and special characters
            Console.OutputEncoding = System.Text.Encoding.UTF8;

            Console.WriteLine("--- Payment Processing System ---");

            // 1. Initialize payment objects (Demonstrating inheritance from IPaymentMethod)
            // I create concrete instances of each payment method
            IPaymentMethod creditCard = new CreditCardPayment();
            IPaymentMethod payPal = new PayPalPayment();
            IPaymentMethod bankTransfer = new BankTransferPayment();

            // 2. Group objects into a list for centralized management
            // This demonstrates the Open/Closed Principle (OCP):
            // New methods can be added to this list without changing the processing logic.
            var methodsToTest = new List<IPaymentMethod> { creditCard, payPal, bankTransfer };

            Random rand = new Random();

            foreach (var method in methodsToTest)
            {
                // 3. DEPENDENCY INJECTION:
                // Initialize the Processor and "inject" the payment object into the constructor
                var processor = new PaymentProcessor(method);

                // Simulate a random transaction amount between 10 and 1000
                decimal randomAmount = rand.Next(10, 1000);

                // Execute the payment processing logic
                processor.Process(randomAmount);
            }
        }
    }
}

```

```
        Console.WriteLine("-----");  
        Console.WriteLine("All transactions processed successfully.");  
        Console.ReadLine(); // Keep the console window open  
    }  
}  
}
```